

Action-Flow Consistency Across Views: Scene-Camera Robustness for Flow-Based VLA Policies

Anonymous Author(s)

Abstract—Vision-language-action (VLA) policies fine-tuned from a fixed scene camera can fail when that camera is displaced, although the task, the objects, and the robot state are unchanged. Fine-tuning on camera-diverse demonstrations is the direct remedy, but such data only exposes the policy to many cameras. It does not state that two images observe one state. This paper adds that statement to the training objective. Two views of one physical state under the same instruction and robot state share the demonstrated action, so for a flow-based VLA we require the action-flow velocity predictions of the two views to agree at shared flow coordinates while each view keeps standard flow matching. The term acts only during training. The deployed policy still receives one scene RGB image, the instruction, and proprioception. On the LIBERO-Plus camera track the objective raises perturbed-camera success by 7.4 percentage points over paired fine-tuning without the consistency term, without costing nominal-camera success under full-support training. It falls below the paired baseline when pair identity is broken, so the gain rests on per-state correspondence. When the training pairs cover only part of the camera-pose range, the gain extends beyond the training support on azimuth, camera distance, and endpoint rotation. It is largest on large azimuth rotations, where test-time view restoration and synthesized-view substitution degrade most. A real-robot study on a three-camera rig shows that the consistency gain persists with cross-view image pairs matched by timestamp and at camera placements not seen during training.

I. INTRODUCTION

Vision-language-action (VLA) policies map RGB images, a language instruction, and the robot state to an action chunk [1, 2], and many current policies generate that chunk with a flow-matching or diffusion head [3–5]. A pretrained VLA is adapted to a task from demonstrations recorded with a wrist camera and one or more scene cameras. The wrist view improves precision near the gripper [6], but its field of view is narrow and it does not see the workspace, so the scene view remains necessary for most manipulation [7, 8]. The two cameras differ in how stable they are. The wrist camera rides on the end effector and is fixed by the mechanical design. The scene camera is placed by hand, its position differs across embodiments, tasks, and sites, and it is bumped, remounted, or moved during ordinary operation [9–11]. A policy fine-tuned from one scene camera can then fail when that camera moves, although the task, the objects, and the robot state are unchanged [12]. When the task-relevant state remains observable, the desired base-frame action does not change with the scene camera, but the policy’s prediction can.

Existing remedies pay for camera robustness at deployment, by giving the policy geometry or the camera pose.

Other methods generate views during training, with their value bounded by the generator, or label every view with its extrinsics (Sec. II). The route with no added cost is to fine-tune on demonstrations recorded from multiple cameras. Such data exposes the policy to many cameras, but standard fine-tuning still supervises each observation independently. We therefore isolate a more specific question: with the paired observations fixed, what is gained by explicitly coupling the action predictions of two views of the same state?

The required correspondence is already present in multi-camera demonstrations. At every timestep, synchronized cameras observe the same physical state and share the command sent to the robot [8, 13, 14]. When actions are expressed in the robot base frame, that command is unchanged by camera placement. We therefore require the two views to predict the same action-flow velocity at the same flow coordinate while retaining standard flow-matching supervision for both views (Fig. 1). Relative to single-view fine-tuning, a pair costs a second forward pass at training time. The constraint needs no extrinsics, leaves the deployed policy unchanged, and takes its pairs from any two cameras that observe one state.

The contributions of this paper:

- a cross-view action-flow objective that uses synchronized multi-camera observations as action-equivalent supervision, without camera extrinsics or any change to the deployed policy;
- a controlled same-data study that isolates the gain of explicit cross-view agreement from paired viewpoint diversity, together with robustness gains at held-out azimuth, camera-distance, and endpoint-rotation poses outside the paired training support;
- hardware validation showing that the same objective remains effective with timestamp-aligned real-camera observations and at unrecorded viewpoints both between the training cameras and beyond the outer cameras.

II. RELATED WORK

Camera geometry supplied at inference. One family makes camera geometry explicit in the deployed policy, through spatial or geometry-aware representations [15], point clouds [16], or orthographic re-projection [17]. Another conditions the policy on the camera pose [18]. Test-time restoration [9] keeps the policy input nominal but uses the target-camera pose to drive a synthesis network at every step. These approaches incorporate camera geometry into the representation or inference pipeline.

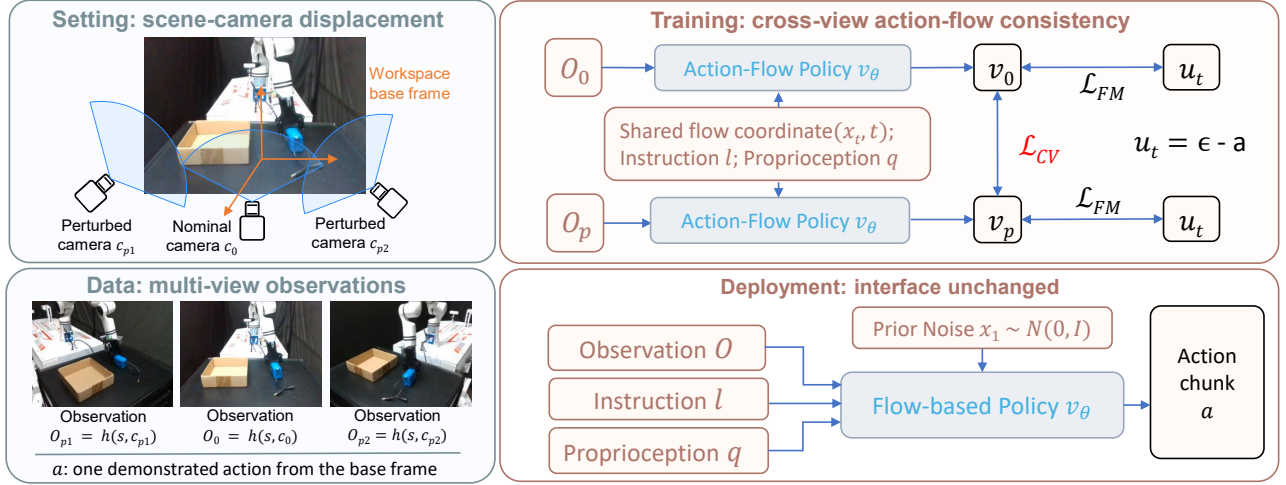


Fig. 1: Scene-camera displacement changes the image of one physical state; training compares the action-flow velocity predictions of the two views at a shared flow coordinate while each view keeps flow matching; the deployed policy is unchanged.

Views generated at training time. Offline viewpoint augmentation synthesizes additional camera views from single-view data with a generative model [11, 19]. Their effectiveness depends on the fidelity of the generated observations; Sec. V-B evaluates this route on the protocol of this paper. Other robustness-oriented post-training methods modify the optimization or representation without synthesizing target views [20, 21].

Invariance imposed by the training objective. Consistency between two views of one input is a standard route to invariance [22–24], and multi-view robot data has been used to learn representations that suppress viewpoint variation, on scene features [25, 26] or on latent actions aligned with the demonstrated action sequence [27]. Representation-level alignment does not directly penalize discrepancies in the action-generating output, which is where we apply the constraint.

Recent VLA methods bring camera robustness closer to action prediction. OC-VLA [28] expresses actions in the camera frame and therefore requires camera-to-robot transformations for training and deployment. OC-VLA++ [29] further enforces cross-view action consistency after mapping predictions from paired views into a common robot frame. CamVLA [10] learns hand-eye geometry from extrinsic supervision and deploys from a single RGB image. SCVC [30] studies cross-view consistency in a world action model and applies it selectively to view-invariant outputs.

Our setting starts from synchronized observations whose actions are already expressed in the same robot base frame. The coordinate-alignment problem is therefore absent, allowing us to isolate a narrower question: with the observations and action supervision fixed, what is gained by explicitly coupling the VLA’s action-generating flow across two views of the same state? We impose that correspondence directly on the flow field without camera-pose labels or coordinate transformations, and Sec. V-A tests whether the resulting constraint extends beyond the camera poses represented by

the training pairs.

III. CROSS-VIEW ACTION-FLOW CONSISTENCY

A flow-based VLA generates an action chunk $a \in \mathbb{R}^{H \times A_{\text{act}}}$ by integrating a conditional velocity field $v_\theta(x_t, t \mid o, l, q)$ from noise, conditioned on the scene image o , the instruction l , and the proprioceptive state q . We take $t = 1$ as pure noise and $t = 0$ as the demonstrated action. A flow coordinate (t, ϵ) with $\epsilon \sim \mathcal{N}(0, I)$ fixes the interpolant $x_t = t\epsilon + (1-t)a$ and the velocity target $u_t = \epsilon - a$, and flow matching minimizes $\|v_\theta(x_t, t \mid o, l, q) - u_t\|^2$ in expectation over coordinates. Every effect of the image on the action passes through this field. The loss fits the field one image at a time, so it never states that two images show the same state.

Multi-camera demonstration data [8, 13, 14] contain exactly this information, since every frame there comes with synchronized images of one state from several cameras. Call the physical state s and write $o = h(s, c)$ for the image of s taken by scene camera c . The demonstrated action a is a command in the robot base frame, so it is the same for every camera. Two images of one state,

$$o_0 = h(s, c_0), \quad o_p = h(s, c_p), \quad (1)$$

therefore have the same target and share (l, q, a) . We call them an action-equivalent pair. The equivalence assumes that both views show what the task needs. A view that hides the target cannot be held to its action. If actions are represented in camera coordinates, comparing the two views requires transforming them through the camera extrinsics. In the robot base frame, both views already share the same action label, so neither c_0 nor c_p needs to be known.

Our objective builds this correspondence into the loss. For each pair we sample K flow coordinates (t_k, ϵ_k) , $k = 1, \dots, K$, and evaluate the field on both images at the same coordinate, $v_0^k = v_\theta(x_{t_k}, t_k \mid o_0, l, q)$ and $v_p^k = v_\theta(x_{t_k}, t_k \mid o_p, l, q)$, restricted to the commanded action dimensions.

Both predictions estimate the same target u_{t_k} , so the residual between them comes from the two images, which differ in the camera. Each coordinate contributes a flow-matching term for both views and a consistency term between them,

$$\mathcal{L}_{\text{FM}}^k = \frac{1}{2} (\|v_0^k - u_{t_k}\|^2 + \|v_p^k - u_{t_k}\|^2), \quad (2)$$

$$\mathcal{L}_{\text{CV}}^k = \|v_p^k - v_0^k\|^2, \quad (3)$$

and the per-pair loss is $\mathcal{L} = \frac{1}{K} \sum_k \mathcal{L}_{\text{FM}}^k + \lambda_{\text{CV}} \frac{1}{K} \sum_k \mathcal{L}_{\text{CV}}^k$ with $\lambda_{\text{CV}} > 0$. Gradients pass through both branches and both predictions come from one network with shared weights. A pair is one training example and costs one forward pass per view. The consistency term adds no forward pass beyond the flow matching of the pair.

Action equivalence can be expressed directly inside the flow trajectory. Under the linear path, a one-step action estimate is $\hat{a} = x_t - t v_\theta(x_t, t \mid o, l, q)$. At a shared flow coordinate, $\|\hat{a}_p - \hat{a}_0\|^2 = t^2 \|v_p^k - v_0^k\|^2$. Thus one-step action consistency applies the same cross-view discrepancy with a t^2 weight, while action-flow consistency acts directly on the velocity field that is integrated to produce the action chunk. We impose the constraint at shared flow coordinates. At deployment the policy is unchanged and still receives a single scene image.

Flow matching alone already trains both views of every pair. What the added term changes can be read off by rewriting the loss. Write $\bar{v}_k = \frac{1}{2}(v_0^k + v_p^k)$ for the mean of the two predictions and $\delta_k = \frac{1}{2}(v_0^k - v_p^k)$ for their disagreement. On the coordinates both terms cover, Eqs. (2) and (3) give

$$\mathcal{L}^k = \|\bar{v}_k - u_{t_k}\|^2 + (1 + 4c\lambda_{\text{CV}}) \|\delta_k\|^2, \quad (4)$$

where c is the ratio of the normalizations of the two terms and equals one when both average over the same coordinates. The first term fits the across-view mean prediction to the shared supervised target. The second measures disagreement between the two views. Flow matching already penalizes this disagreement with weight one, and the consistency term increases only that weight while leaving u_{t_k} unchanged. Because state, action, instruction, proprioception, and flow coordinate are shared within a pair, δ_k isolates the part of the action-flow prediction that changes with the observation. The objective therefore reweights view-conditioned variation inside the supervised action-generating field rather than introducing a new action target.

This decomposition relies on the two predictions sharing one supervised target. To see what changes when the partner comes from another state of the same task, let the partner of state s be an independent state s' drawn from the same distribution, each branch evaluated at its own flow coordinate. Expanding the square and using independence, the population objective becomes

$$\mathbb{E}_{s,s'} \|v_p(s') - v_0(s)\|^2 = \text{Tr Var}_s v_p + \text{Tr Var}_s v_0 + \|\mathbb{E}_s v_p - \mathbb{E}_s v_0\|^2. \quad (5)$$

where expectations and variances run over states, each with its own coordinate. Every term is nonnegative, and the objective is at its minimum when neither prediction varies

with the sampled state and flow coordinate. Matched pairs suppress variation inside an action-equivalent set, where state, action, instruction, proprioception, and flow coordinate are fixed. Shuffled pairs instead penalize variation that the supervised objective must preserve across states. Pairing therefore tells the regularizer which variation is nuisance and which variation carries the action. Pairing across states should harm the policy.

IV. ANALYSIS OF THE CONSISTENCY TERM

The analysis above yields two empirical questions. First, does increasing the penalty on cross-view disagreement improve robustness over exactly the same paired supervision? Second, does that gain depend on preserving the state correspondence of the pair? Secs. IV-A and IV-B test these two consequences under the same training recipe. Sec. IV-C then applies the consistency term to only a fraction of the demonstration episodes and measures how much of the gain survives.

All simulation results use the LIBERO-Plus camera track [12]. The track perturbs the scene camera of the LIBERO tasks along three families, camera distance, spherical position (azimuth, elevation, and their compound), and endpoint rotation (1,599 camera-task instances $\times$ 3 rollouts = 4,797 trials per policy). Nominal-camera success is measured on LIBERO [31] (40 tasks $\times$ 50 trials = 2,000). Pairs come from the LIBERO demonstrations: each stored simulator state is rendered under the nominal camera and under one perturbed camera sampled from the same three families (2,000 episodes, 40 tasks, 338,575 pairs). The perturbed camera is drawn once per episode and held fixed for all its frames, so every episode is recorded as if by a two-camera rig.

We compare four training regimes. Nominal-only uses only the nominal scene-camera observations. Mixed-camera fine-tunes on camera-diverse observations but treats every observation independently. Paired data only organizes the nominal and perturbed observations of each state as a synchronized pair and applies standard flow matching to both views. Paired + consistency uses exactly the same paired batches and adds only the cross-view term in Eq. (3). Thus, the last two settings isolate explicit cross-view coupling from the effect of seeing the paired viewpoints themselves.

The main comparison fine-tunes $\pi_{0.5}$ [4] for 10,000 steps with AdamW and a cosine learning-rate schedule. Paired runs use batches of 192 synchronized pairs, 224×224 images, and action horizon $H=10$. We apply independent color jitter to the two views and no spatial augmentation. The final consistency configuration uses $K=2$, $\lambda_{\text{CV}}=0.10$, and $t \sim \text{Beta}(2, 3)$, with λ_{CV} ramped from zero over the first 500 steps. Paired data only uses the same paired batches, flow-coordinate sampling, optimization, and training budget, with $\lambda_{\text{CV}}=0$.

Flow matching averages over the padded action width of 32, whereas the consistency term covers the 7 commanded dimensions, giving $c=32/7$ in Eq. (4). The simulator runs at 20 Hz, and the policy executes 5 of the 10 predicted

TABLE I: Nominal-camera and LIBERO-Plus camera-track success (%). $\pm$: standard deviation over three training seeds; rows without it are one training seed.

Method	Nominal	Camera track			
		All	Camera distance	Spherical position	Endpoint rotation
Nominal-only	91.1	16.8	1.1	13.2	45.7
Mixed-camera	78.7	74.7	68.4	75.7	78.2
Paired data only	93.6 $\pm$ 0.4	79.8 $\pm$ 0.8	72.7 $\pm$ 0.2	79.9 $\pm$ 1.4	87.2 $\pm$ 0.7
Paired + consistency	94.2 $\pm$ 0.8	87.2 $\pm$ 0.4	81.9 $\pm$ 1.0	87.9 $\pm$ 0.5	90.6 $\pm$ 0.9
shuffled pairs	50.8	25.8	19.4	25.7	33.3
t^2 -weighted term	94.3	86.7	81.9	87.5	89.0
shared color jitter	94.9	87.2	82.7	87.7	90.4

actions before replanning. We mask the wrist camera because it is unchanged by scene-camera perturbations and would otherwise weaken the benchmark’s isolation of the scene channel [12].

A. The consistency term improves on paired data

Table I gives the main comparison in its first four rows and then reports targeted controls. The shuffled, t^2 -weighted, and shared-jitter rows use the Paired + consistency recipe and one base checkpoint. Nominal-only fine-tuning succeeds under its own camera and drops to 16.8% once the camera moves. Mixed-camera fine-tuning recovers most of the camera track from viewpoint diversity alone. Organizing the training observations as synchronized pairs raises camera-track success further to 79.8% while restoring nominal-camera performance.

The controlled comparison is between the two paired rows. They use the same observations, sampler, batch construction, optimization, flow-coordinate sampling, and training budget, and differ only in whether the cross-view term is enabled. Consistency raises camera-track success from 79.8% to 87.2% across three seeds. The lowest of its three training seeds exceeds the highest of the three baseline seeds. The gain appears in all three perturbation families, while nominal-camera success remains within the variation across training seeds.

An earlier single-coordinate sweep showed similar performance at $\lambda_{CV}=0.05$ and 0.10, while larger weights reduced both camera-track and nominal success. The main Paired + consistency result uses the final $K=2$, Beta(2, 3) configuration with $\lambda_{CV}=0.10$.

The next two controls probe where the consistency is applied and whether independent photometric variation contributes to the gain. The t^2 -weighted row places consistency on one-step actions (Sec. III), with $\lambda_{CV}=0.50$ so that its mean weight under Beta(2, 3) matches the proposed term. Relative to the same training seed of Paired + consistency, it is 0.2 points lower on the camera track and 0.8 lower on the nominal camera. The shared-jitter row is 0.3 points higher on the track and 0.2 lower on the nominal camera. Both differences are smaller than the variation across the three main training seeds. The t^2 control shows that action-space and velocity-space placements perform similarly. We use the action-flow form because it acts directly on the supervised

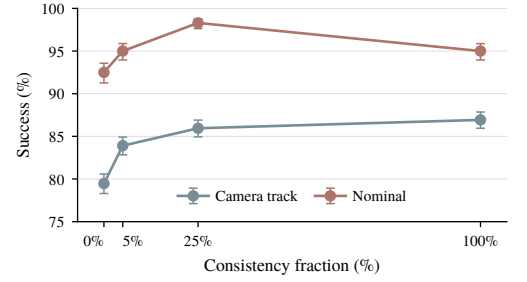


Fig. 2: Success versus the fraction of training episodes with consistency (one seed; Wilson 95% intervals).

velocity field at the shared flow coordinate. Independent color jitter is likewise not responsible for the improvement. The improvement comes from enforcing agreement between action-equivalent views.

B. The gain depends on action-equivalent pairing

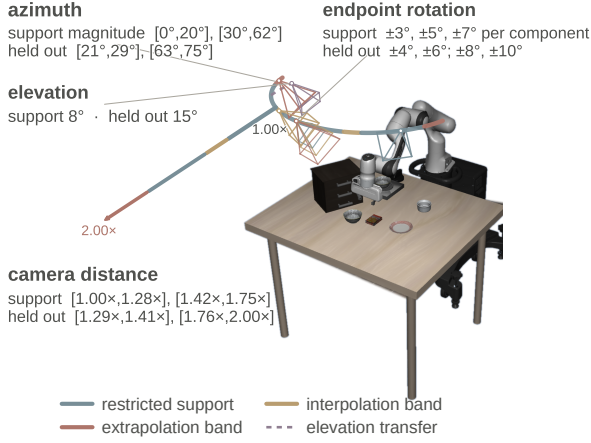
Table I attributes the gain to the consistency term. It does not say whether the term acts through per-state correspondence or as a generic smoothing of the velocity field, which any two views of the task would provide. Eq. (5) predicts that the two cases differ, and the shuffled-pair control tests that prediction. For a batch of N pairs and a derangement π (a permutation of $\{1, \dots, N\}$ with no fixed point), the consistency partner of state i is taken from state $\pi(i)$,

$$\mathcal{L}_{CV,shuf}^k = \|v_{p,\pi(i)}^k - v_{0,i}^k\|^2, \quad (6)$$

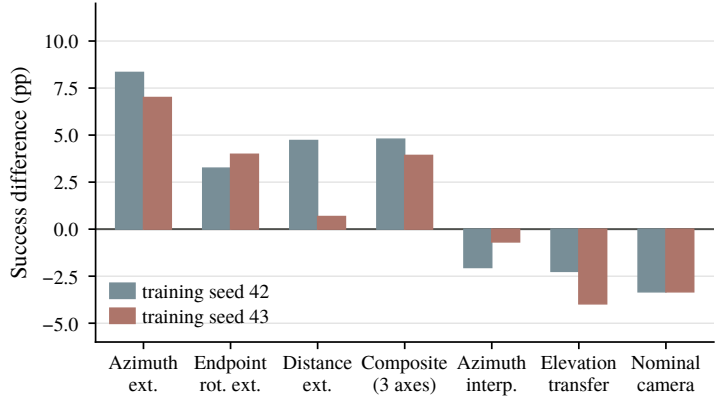
where each prediction is evaluated at the flow coordinate of its own state, so every supervised label stays correct. The control breaks state identity and, with it, the shared coordinate of Eq. (3), since a state’s coordinate is built from its own action. Because the shuffled partner carries its own action and flow coordinate, shuffling changes state identity and the shared flow coordinate together. The control therefore tests whether non-corresponding views from the same task can serve as consistency partners. The shuffled row of Table I gives the result. Camera-track success falls to 25.8%, and nominal success falls to 50.8%. Shuffling changes the regularizer from agreement within one state to agreement across different states. As Eq. (5) predicts, this penalizes state-dependent variation that the supervised objective must preserve. The gain therefore depends on preserving the matched action-equivalent structure of the pair.

C. A small consistency fraction suffices

Eq. (4) shows that the consistency term reweights view disagreement. We therefore ask how broadly this signal must be applied across the paired dataset. The observation pool, optimizer budget, and schedule stay fixed. Every episode still enters training as a pair, exactly as in the Paired data only row, while the consistency term is enabled for a fraction of source episodes, stratified by task. Fig. 2 gives the curve over one training seed with Wilson 95% intervals over evaluation rollouts. Its two endpoints are one of the three training seeds



(a) Restricted support and held-out bands per axis.



(b) Paired + consistency minus Paired data only.

Fig. 3: Restricted-support analysis: (a) the retained pairs and the held-out bands on each axis; (b) camera-track success of Paired + consistency minus Paired data only per evaluation group, one bar per training seed;

behind the Paired data only and Paired + consistency rows of Table I. Applying consistency to 5% of the episodes recovers about 60% of the gain between the two endpoints, and 25% recovers about 85%. Nominal success never falls below its value at 0%.

V. ROBUSTNESS BEYOND THE PAIRED SUPPORT

In Sec. IV the perturbed camera of every pair is drawn from the same distribution of poses that the evaluation track covers. Collecting demonstrations from that many viewpoints is impractical in deployment, so a deployed policy will meet camera poses that no pair covered. Sec. V-A trains on pairs restricted to part of each perturbation axis and evaluates on the withheld part, and asks whether the consistency term extrapolates beyond the pairs it saw. Sec. V-B asks whether generated observations can substitute for collected views at the missing poses, using test-time restoration [9] and training-time synthesis [19] on the same protocol. The two results meet on one axis. The consistency term gains most on large azimuth rotations, and that is where generated views fail.

A. The gain extends to unseen viewpoints

We restrict the support of the paired training data. Along each perturbation axis we hold out two bands of camera poses, one in the interior of the axis and one beyond its outer edge, and keep the rest (Fig. 3a). For elevation, the perturbed side of each training pair is fixed at 8° relative to the nominal 0° view, while the 15° elevation is held out entirely. On the restricted pairs we retrain the paired baseline (Paired data only in Table I) and the consistency term (Paired + consistency) with two training seeds, and we evaluate both on the full camera track. Each evaluation camera then belongs to one of four groups: inside the restricted support, in an interior held-out band (interpolation), beyond the outer edge (extrapolation), or at the held-out elevation (transfer). The spherical family is split into azimuth, elevation, and their

compound. The extrapolation bands hold 300, 297, and 678 evaluation rollouts per policy on azimuth, camera distance, and endpoint rotation, and the three-axis extrapolation composite pools all 1,275.

The consistency term improves the policy at viewpoints it never trained on. In both seeds it beats the paired baseline at extrapolation on azimuth, endpoint rotation, and camera distance, and the three-axis composite is positive (Fig. 3b). The margin is largest on azimuth. Inside the restricted support the two objectives are close, and on azimuth interpolation the baseline is slightly ahead. The term adds little where the training pairs already cover the viewpoint and adds most beyond them.

Elevation is the exception. Although the training pairs contain a $0^\circ \leftrightarrow 8^\circ$ elevation displacement, the consistency model performs worse than the paired baseline at the held-out 15° elevation in both seeds. Thus, the positive extrapolation observed on azimuth, camera distance, and endpoint rotation does not extend uniformly to every camera factor. The same restricted-support setting also lowers nominal-camera success in both seeds, a cost absent under full-support training (Table I).

B. Generated views on the held-out camera bands

We next test whether generated observations can substitute for collected views at the held-out poses. Table II measures two such routes on the same three bands, one per axis.

Test-time restoration keeps the policy frozen and places a view synthesis model in front of it. We instantiate this route with the LVSM restoration model released with Any-CamVLA [9]. At every control step it takes the perturbed scene image, the wrist image, and the calibrated camera extrinsics, and it synthesizes the nominal scene view. The policy receives that view in place of the perturbed image, and nothing else changes. Table II shows the route lifting the nominal-only policy on camera distance and endpoint rotation and leaving it near zero on azimuth. Fig. 4(a)

TABLE II: Success (%) on the three extrapolation bands of Fig. 3a, training seed 42 throughout. The first block trains on the restricted support; the second restores the nominal view at test time; the third trains on the full pose range and attempts to replace each perturbed render with a generated frame, with nominal fallback under the quality guard.

Configuration	Azimuth	Camera distance	Endpoint rotation
<i>Collected pairs, restricted support (Sec. V-A)</i>			
Paired data only	41.0	53.5	83.6
Paired + consistency	49.3	58.2	86.9
<i>Test-time restoration (LVSM from AnyCamVLA)</i>			
Nominal-only, alone	1.3	0.0	38.1
Nominal-only, restored	3.7	45.5	61.5
Paired data only, restored	7.3	22.2	75.5
Paired + consistency, restored	1.7	24.6	73.5
<i>Training-time substitution (VISTA), full support</i>			
Paired data only, real renders	64.7	59.6	86.3
Paired data only, substituted	3.7	5.1	76.0

illustrates a representative failure: the layout survives the largest azimuth offset but the task objects do not. On every axis the restored nominal policy stays below the two restricted-support policies, which never saw the band either and receive one perturbed image. Placed in front of those two policies, restoration still lowers both, and the margin of the consistency term does not survive the transformed inputs. The route also needs the camera extrinsics at every control step, raises one policy query from 59.8 ms to 151.2 ms on an NVIDIA RTX Pro 6000, and holds 2.4 GiB of additional GPU memory. The consistency term adds none of this at inference.

Training-time synthesis generates the perturbed views before training, so the deployed policy is untouched. We run VISTA [19] with its ZeroNVS generator [32] on the pair dataset of Table I. Every nominal image, label, and pose is kept, each perturbed render is replaced by a frame generated at the recorded pose, and the quality guard retries generation and falls back to the nominal image if no sample passes VISTA’s similarity check. The Paired-data-only recipe is trained on the substituted data and set against the same recipe and training seed on the real renders. Table II shows three outcomes. On azimuth the quality guard falls back to the nominal image for 95.0% of the generated frames, so the substituted data provide little variation along that axis and the resulting policy scores near zero. On camera distance the fallback rate is only 1.1%, yet the substituted policy still scores near zero. Fig. 4(b) illustrates a representative mismatch: the background recedes with the target pose while the arm and gripper keep the size and position of the source, altering the gripper-object relation seen by the policy. Endpoint rotation retains most of the real-render success, with a 26.0% fallback rate. Across both routes, the generated observations can fail to preserve the manipulation geometry associated with the requested camera pose. For training-time substitution, collected restricted-support pairs outperform synthesized full-support pairs on all three bands. The generation also costs about 596 GPU-hours for the

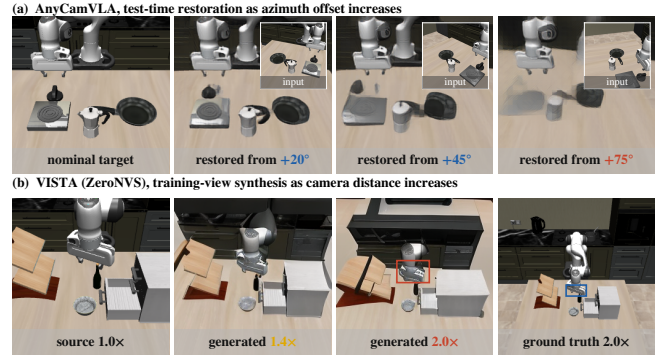


Fig. 4: Views generated by test-time restoration (a) and by training-time synthesis (b).

338,575 frames on RTX Pro 6000, against about 4.5 GPU-hours to render the same pairs from the stored simulator states, and on hardware the pairs are a by-product of the camera rig (Sec. VI).

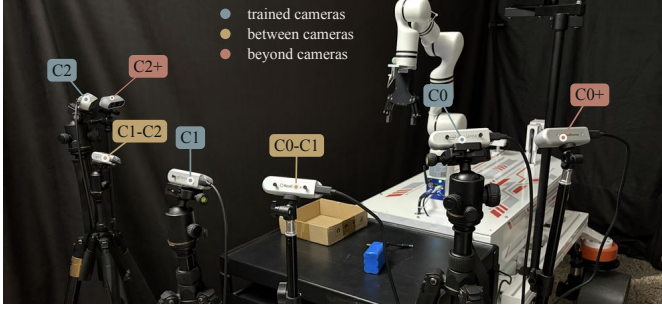
Both routes degrade most on azimuth, which is also the axis on which the consistency term gains most (Fig. 3b, Table II). Azimuth rotation produces the largest image change in this protocol: it shifts the field of view and exposes surfaces absent from the source observation. The examples in Fig. 4 show that generated views can preserve coarse scene layout while altering task-relevant object or gripper geometry. At these pose offsets, neither generated-view route matches the performance obtained from collected views. The consistency term uses observed views directly: it needs a second camera and a shared clock, produces no image, and adds no generation cost before training and no synthesis cost at inference (Sec. VI).

VI. REAL-ROBOT EVALUATION

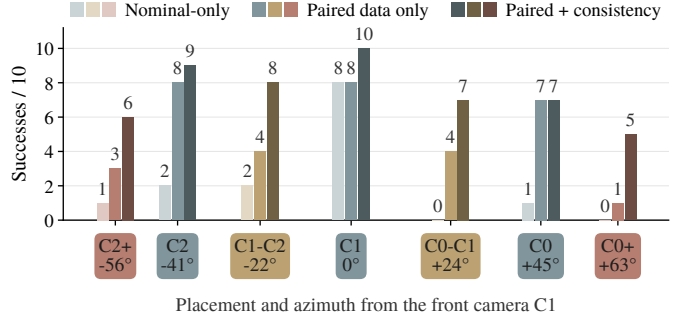
Simulation makes action-equivalent pairs exact: two images are rendered from one stored state. Hardware removes that convenience. We therefore ask whether the same supervision remains useful when pairs come from ordinary timestamp-aligned cameras, and when the paired support consists of only a few fixed viewpoints. The latter gives a hardware counterpart to Sec. V-A: evaluation can move both between the recorded cameras and beyond their outermost views.

We use an RM-75 7-DoF arm with a two-finger gripper and three RealSense D435i cameras, C0–C2 (Fig. 5a). The task is to place a battery into a box from randomized initial object positions. We collect 98 teleoperated demonstrations while recording all three scene cameras. There is no hardware trigger; cross-view observations are paired by timestamp, with a maximum offset of 67 ms, and paired images inherit the same base-frame action label.

Nominal-only uses the front camera C1. Paired data only and Paired + consistency use exactly the same multi-camera demonstrations and timestamp-aligned pairs, and differ only in whether the cross-view term is enabled. The hardware runs follow the same 10,000-step training recipe and the same consistency configuration used in simulation: $K=2$, $\lambda_{CV}=0.10$, and $t \sim \text{Beta}(2, 3)$.



(a) Camera placements.



(b) Success on shared object layouts.

Fig. 5: Real-robot evaluation under camera displacement. Three cameras provide training views and four placements are held out; each placement uses the same 10 object layouts.

Evaluation keeps the same 10 object layouts and uses seven camera placements: the three recorded cameras, two unseen viewpoints between them, and two beyond the outer cameras. The held-out placements vary azimuth while keeping the remaining camera geometry within the training range. A ChArUco board is used only to measure the positions shown in Fig. 5; calibration is not an input to either training or deployment.

The hardware reproduces the camera sensitivity seen in simulation. Nominal-only remains effective near C1 but deteriorates sharply once the scene camera moves. Multi-camera supervision largely fixes the cameras that supplied training data: Paired data only succeeds on 23 of the 30 rollouts from C0–C2. Its robustness does not, however, fill the gaps between those cameras. Across the four held-out placements it succeeds on only 12 of 40 rollouts. With exactly the same demonstrations, Paired + consistency raises this to 26 of 40 while also reaching 26 of 30 on the recorded cameras. The advantage appears both between cameras and beyond the two outer cameras (Fig. 5b). On the shared held-out trials, the consistency model converts 15 baseline failures into successes and reverses only once.

The separation between recorded and unrecorded viewpoints is the key result. Paired data only performs well at the three cameras that supplied demonstrations but leaves a pronounced gap at the four held-out placements. Adding cross-view consistency closes much of that gap while retaining performance on the recorded cameras. The restricted-support simulation experiment (Sec. V-A) likewise showed its largest extrapolation gain along azimuth. In hardware, azimuth is the camera factor evaluated outside the recorded support. Unlike the simulator pairs, the hardware observations are matched by timestamp rather than exact state rerendering. The gain therefore persists under the synchronization accuracy of this setup. Camera extrinsics remain unnecessary because both views inherit the same base-frame action label.

The remaining failures expose a different limitation. When the scene view does not reveal enough geometry for grasp alignment, all three policies can fail. These failures are consistent with limited scene-view observability and are not resolved by cross-view consistency alone. In practical

systems, a wrist camera could provide complementary close-range information, but multi-camera sensor fusion is outside the scope of this study. Our experiments deliberately isolate the scene-camera channel to measure robustness to its displacement.

VII. CONCLUSION

Multi-camera demonstrations contain a direct supervision signal for camera robustness because synchronized views of one state share one demonstrated action. We build that shared action into the flow-matching objective of a VLA by enforcing agreement between the action-flow predictions of the two views. The term needs no camera extrinsics and leaves the deployed policy unchanged. On the LIBERO-Plus camera track, it raises perturbed-camera success over the same pairs trained without the term while preserving nominal-camera success. Shuffling the consistency partners collapses performance, showing that the benefit depends on preserving the matched action-equivalent structure of the pair. In the fraction study, applying consistency to only a subset of training episodes recovers most of the full gain.

Under restricted-support training, the gain extends to held-out azimuth, camera-distance, and endpoint-rotation poses. Elevation provides a counterexample: training on $0^\circ \leftrightarrow 8^\circ$ pairs does not improve the held-out 15° view. The evaluated generated-view alternatives degrade most on azimuth, where the consistency gain is largest. On a three-camera rig with cross-view image pairs matched by timestamp, the consistency term also improves success at unrecorded viewpoints between the training cameras and beyond their outer range. The method requires multiple views that observe the same task-relevant state, and the experiments cover one flow-based backbone, $\pi_{0.5}$.

REFERENCES

- [1] A. Brohan, N. Brown, J. Carbajal *et al.*, “RT-2: Vision-language-action models transfer web knowledge to robotic control,” in *CoRL*, 2023.
- [2] M. J. Kim, K. Pertsch, S. Karamcheti *et al.*, “Open-VLA: An open-source vision-language-action model,” in *CoRL*, 2024.

- [3] K. Black, N. Brown, D. Driess *et al.*, “ π_0 : A vision-language-action flow model for general robot control,” *arXiv:2410.24164*, 2024.
- [4] Physical Intelligence, K. Black, N. Brown *et al.*, “ $\pi_{0.5}$: A vision-language-action model with open-world generalization,” *arXiv:2504.16054*, 2025.
- [5] C. Chi, S. Feng, Y. Du *et al.*, “Diffusion policy: Visuomotor policy learning via action diffusion,” in *RSS*, 2023.
- [6] K. Hsu, M. J. Kim, R. Rafailov *et al.*, “Vision-based manipulators need to also see from their hands,” in *ICLR*, 2022.
- [7] T. Li, J. Zhang, S. Wang *et al.*, “LIBERO-Occ: Evaluating and improving vision-language-action models under scene-induced occlusion via viewpoint imagination,” *arXiv:2606.10862*, 2026.
- [8] O. Rayyan, J. Abanes, M. Hafez *et al.*, “MV-UMI: A scalable multi-view interface for cross-embodiment learning,” *arXiv:2509.18757*, 2025.
- [9] H. Heo, S. Woo, S. M. Kim *et al.*, “AnyCamVLA: Zero-shot camera adaptation for viewpoint robust vision-language-action models,” in *IROS*, 2026.
- [10] W. Li, X. Jiang, Q. Qian *et al.*, “From fixed to free cameras: Calibration-free view-robust vision-language-action model,” *arXiv:2607.05396*, 2026.
- [11] L. Y. Chen, C. Xu, K. Dharmarajan *et al.*, “RoVi-Aug: Robot and viewpoint augmentation for cross-embodiment robot learning,” in *CoRL*, 2024.
- [12] S. Fei, S. Wang, J. Shi *et al.*, “LIBERO-Plus: In-depth robustness analysis of vision-language-action models,” *arXiv:2510.13626*, 2025.
- [13] A. Khazatsky, K. Pertsch, S. Nair *et al.*, “DROID: A large-scale in-the-wild robot manipulation dataset,” in *RSS*, 2024.
- [14] H.-S. Fang, H. Fang, Z. Tang *et al.*, “RH20T: A comprehensive robotic dataset for learning diverse skills in one-shot,” in *ICRA*, 2024.
- [15] D. Qu, H. Song, Q. Chen *et al.*, “SpatialVLA: Exploring spatial representations for visual-language-action model,” *arXiv:2501.15830*, 2025.
- [16] C. Li, J. Wen, Y. Peng *et al.*, “PointVLA: Injecting the 3D world into vision-language-action models,” *arXiv:2503.07511*, 2025.
- [17] I. Singh, A. Goyal, S. Birchfield *et al.*, “OG-VLA: Orthographic image generation for 3D-aware vision-language action model,” *arXiv:2506.01196*, 2025.
- [18] T. Jiang, J. Ji, X. Tan *et al.*, “Do you know where your camera is? view-invariant policy learning with camera conditioning,” *arXiv:2510.02268*, 2025.
- [19] S. Tian, B. Wulfe, K. Sargent *et al.*, “View-invariant policy learning via zero-shot novel view synthesis,” *arXiv:2409.03685*, 2024.
- [20] J. Guo, Z. Wu, C. Tu *et al.*, “RobustVLA: On robustness of vision-language-action model against multi-modal perturbations,” *arXiv:2510.00037*, 2025.
- [21] Y. Fu, C. Zhang, S. Gong *et al.*, “StableVLA: Towards robust vision-language-action models without extra data,” *arXiv:2605.18287*, 2026.
- [22] S. Laine and T. Aila, “Temporal ensembling for semi-supervised learning,” in *ICLR*, 2017.
- [23] A. Tarvainen and H. Valpola, “Mean teachers are better role models: Weight-averaged consistency targets improve semi-supervised deep learning results,” in *NeurIPS*, 2017.
- [24] J.-B. Grill, F. Strub, F. Altché *et al.*, “Bootstrap your own latent: A new approach to self-supervised learning,” in *NeurIPS*, 2020.
- [25] Y. Seo, J. Kim, S. James *et al.*, “Multi-view masked world models for visual robotic manipulation,” in *ICML*, 2023.
- [26] J.-C. Pang, N. Tang, K. Li *et al.*, “Learning view-invariant world models for visual robotic manipulation,” in *ICLR*, 2025.
- [27] Y. Jeong, J. Chun, and T. Kim, “Learning to act robustly with view-invariant latent actions,” in *CVPR*, 2026.
- [28] T. Zhang, H. Duan, H. Hao *et al.*, “Grounding actions in camera space: Observation-centric vision-language-action policy,” in *AAAI*, 2026.
- [29] T. Zhang, Z. Gong, Z. Yang *et al.*, “OC-VLA++: Monocular geometry-guided cross-view consistency for viewpoint-robust robotic manipulation,” *arXiv:2608.01066*, 2026.
- [30] B. Huang, B. Wei, Y. Cai *et al.*, “Selective cross-view consistency for world action models: Held-out viewpoint robustness without test-time camera information,” *arXiv:2608.21402*, 2026.
- [31] B. Liu, Y. Zhu, C. Gao *et al.*, “LIBERO: Benchmarking knowledge transfer for lifelong robot learning,” in *NeurIPS*, 2023.
- [32] K. Sargent, Z. Li, T. Shah *et al.*, “ZeroNVS: Zero-shot 360-degree view synthesis from a single image,” in *CVPR*, 2024.